

RainDrop: Reward Distribution Protocol

Zahnentferner ✉

The Stable Order

Sarthak Dengre ✉

The Stable Order

Abstract

This paper presents the RainDrop protocol for efficient proper reward distribution to token stakers: stakers receive rewards proportionally to their staked balances; and each reward is only distributed to stakers who were already staking before the reward is distributed.

2012 ACM Subject Classification Applied computing → Economics

Keywords and phrases blockchain, cryptocurrency, staking, reward distribution

Acknowledgements We thank the community of the Stability Nexus for their feedback and support.

1 Introduction

Distributing rewards to token holders is a key financial operation. Note that dividend distribution, which is widely used in the legacy financial world, is a special case of reward distribution, where the reward is a fiat currency and the token is a company share.

Here we address the more general case where:

1. the held asset can be any fungible token;
2. the rewards to be distributed can be of any fungible token as well, possibly different from the held token;
3. the distribution is to be made automatically via smart contracts on a blockchain;
4. the distribution ought to be *proper*, in the sense that each token holder should receive an amount of rewards proportional to their token balances.

Surprisingly, this problem is not trivial. The first challenge lies in the fact that we cannot do the distribution to all token holders in a single transaction. The cost of this transaction would be unbounded and, for a sufficiently large number of token holders, could exceed the cost limit for the transaction to be included in a block. In other words, such an operation would be linear in the number of token holders. This challenge is solved with the idea of *staking*. Instead of performing a single linear distribution operation, the following is done: token holders are required to *stake* their tokens (i.e. deposit their tokens into the smart contract, so that the smart contract knows their balances); the rewards are *distributed* into a common pool; and then each token holder is expected to *claim* their portion of rewards from the pool. Each of these actions is a constant-time operation. So, staking turns a single linear-time transaction into a linear number of constant-time transactions.

The second challenge, however, is how to implement these actions in such a way that the amounts received by the stakers are correctly proportional to their staked balances. Naive implementations do not work.

Here is an example. Suppose that Alice and Bob staked 100 ABC tokens each and 50 XYZ tokens are distributed into a common reward pool. Alice then claims her portion (25 XYZ) from the pool. Then a second distribution of 50 XYZ tokens is made into the pool. Now the pool has 75 XYZ. Suppose that Alice claims again. She would now get 37.5 XYZ. In total, she has already received 62.5 XYZ, which is more than the 50 XYZ that she should have received.

The approach illustrated in the example above is obviously too naive. Popular staking reward distribution contracts¹ have mechanisms to track that Alice has already claimed and reduce her share of the pool, so that she would be able to claim a smaller amount from the second distribution. But such mechanisms can be partially circumvented by unstaking and staking again with a different account.

Unlike such popular but flawed staking reward distribution contracts, the RainDrop Protocol systematically tracks each distribution (*drop*) independently and with sufficient data to ensure that every staker can only claim the correct amount from each drop.

2 Protocol Description

Each RainDrop vault allows user to stake a single fungible token and receive rewards in any fungible token. Users can perform 5 actions:

1. **Stake:** a user deposits an amount of the vault's token into the vault and the user's entry point in the drop timeline is registered.
2. **Unstake:** the user withdraws their staked tokens from the vault.
3. **Drop:** any user can distribute any fungible token by creating a drop.
4. **Claim:** a user can claim their proportional share of a drop that occurred after their staking event.
5. **Withdraw:** after a drop's claim deadline, the original drop creator can withdraw all unclaimed tokens.

3 Protocol Specification

The *vault state* at time t is defined as:

$$V(t) = (S_{total}(t), S_{user}(t), E_{user}(t), D(t), k(t), H(t)) \quad (1)$$

where:

- $S_{total}(t) \in \mathbb{R}^+$: Total staked amount
- $S_{user}(t) : U \rightarrow \mathbb{R}^+$: User stake mapping
- $E_{user}(t) : U \rightarrow \mathbb{N}$: User entry drop counter
- $D(t) = \{d_1, d_2, \dots, d_k\}$: Sequence of drops
- $k(t) \in \mathbb{N}$: Current drop counter
- $H(t) : U \times \mathbb{N} \rightarrow \{0, 1\}$: Claim history mapping

And the *balance state* is defined as:

- $B_{user}(u, \tau) \in \mathbb{R}^+$: External balance of user u for token τ
- $B_{vault}(\tau) \in \mathbb{R}^+$: Vault balance for token τ
- τ_{stake} : The vault's designated staking token type

A *drop* d_i is a 5-tuple:

$$d_i = (A_i, \tau_i, S_i, T_i, C_i) \quad (2)$$

where:

¹ e.g. Pancake Swap's Masterchef v2 (<https://bscscan.com/address/0xa5f8C5Dbd5F286960b9d90548680aE5ebFf07652#code>) and SushiSwap's Masterchef v1 (<https://etherscan.io/address/0xc2edad668740f1aa35e4d8f227fb8e17dca888cd#code>)

- $A_i \in \mathbb{R}^+$: Total amount dropped
- $\tau_i \in \Omega$: Token address
- $S_i \in \mathbb{R}^+$: Total staked at drop time
- $T_i \in \mathbb{R}^+$: Claim deadline (timestamp)
- $C_i \in \mathbb{R}^+$: Total amount already claimed from this drop

3.1 Protocol Operations

RainDrop's operations are precisely specified in the following subsections.

3.1.1 Stake

The *stake* operation $\sigma : U \times \mathbb{R}^+ \rightarrow V \rightarrow V$ is defined as:

$$\sigma(u, n)(V) = \begin{cases} S_{total} \leftarrow S_{total} + n \\ S_{user}(u) \leftarrow S_{user}(u) + n \\ E_{user}(u) \leftarrow k \\ B_{user}(u, \tau_{stake}) \leftarrow B_{user}(u, \tau_{stake}) - n \\ B_{vault}(\tau_{stake}) \leftarrow B_{vault}(\tau_{stake}) + n \end{cases} \quad (3)$$

3.1.2 Unstake

The *unstake* operation $\mu : U \times \mathbb{R}^+ \rightarrow V \rightarrow V$ is defined as:

$$\mu(u, n)(V) = \begin{cases} S_{total} \leftarrow S_{total} - n \\ S_{user}(u) \leftarrow S_{user}(u) - n \\ B_{user}(u, \tau_{stake}) \leftarrow B_{user}(u, \tau_{stake}) + n \\ B_{vault}(\tau_{stake}) \leftarrow B_{vault}(\tau_{stake}) - n \end{cases} \quad (4)$$

Preconditions: $S_{user}(u) \geq n$ (user has sufficient staked balance)

3.1.3 Drop

The *drop* operation $\delta : U \times \mathbb{R}^+ \times \Omega \times \mathbb{R}^+ \rightarrow V \rightarrow V$ is defined as:

$$\delta(u, A, \tau, T)(V) = \begin{cases} D \leftarrow D \cup \{(A, \tau, S_{total}, T, 0)\} \\ k \leftarrow k + 1 \\ \forall u' \in U : H(u', k) \leftarrow 0 & \text{initialize claim history for new drop} \\ B_{user}(u, \tau) \leftarrow B_{user}(u, \tau) - A \\ B_{vault}(\tau) \leftarrow B_{vault}(\tau) + A \end{cases} \quad (5)$$

3.1.4 Claim

The *claim* operation $\gamma : U \times \mathbb{N} \rightarrow V \rightarrow \mathbb{R}^+$ is defined as:

$$\gamma(u, i)(V) = \begin{cases} H(u, i) \leftarrow 1 \\ C_i \leftarrow C_i + q \\ B_{user}(u, \tau_i) \leftarrow B_{user}(u, \tau_i) + q \\ B_{vault}(\tau_i) \leftarrow B_{vault}(\tau_i) - q \end{cases} \quad (6)$$

where:

$$q = \begin{cases} \frac{S_{user}(u) \times A_i}{S_i} & \text{if } i > E_{user}(u) \text{ and } H(u, i) = 0 \\ 0 & \text{otherwise} \end{cases} \quad (7)$$

3.1.5 Withdraw

The *withdraw* operation $\zeta : U \times \mathbb{N} \rightarrow V \rightarrow \mathbb{R}^+$ is defined as:

$$\zeta(u, i)(V) = \begin{cases} B_{user}(u, \tau_i) \leftarrow B_{user}(u, \tau_i) + R_i \\ B_{vault}(\tau_i) \leftarrow B_{vault}(\tau_i) - R_i \\ C_i \leftarrow C_i - R_i \end{cases} \quad (8)$$

where:

$$R_i = \begin{cases} A_i - C_i & \text{if } u = P_i \text{ and } t_{now} > T_i \\ 0 & \text{otherwise} \end{cases} \quad (9)$$

4 Related Work

The RainDrop protocol improves various reward distribution mechanisms. Below we mention and compare some of the most influential ones in the conception of RainDrop.

- **Hodlcoin:** hodlCoin [16] is a staking protocol that introduced the concept of unstaking fee and proved that the price of the staking token never decreases. Like RainDrop, hodlCoin's reward distribution is also efficient and proper, and hodlCoin is even significantly simpler than RainDrop. But, in hodlCoin, the tokens distributed as rewards must be the same as the staked token. This limitation prevents cross-token incentivization, which RainDrop addresses by supporting arbitrary reward tokens.
- **Popular DEX Staking Contracts:** PancakeSwap's MasterChef v2 and SushiSwap's MasterChef v1 are widely adopted staking contracts that attempt to solve proportional reward distribution. However, these systems suffer from the gaming vulnerability mentioned in the introduction: users can unstake and re-stake with different accounts to manipulate their reward eligibility. RainDrop prevents this by binding reward eligibility to specific drop timestamps rather than dynamic staking status.
- **Other Staking Protocols:** The DeFi ecosystem includes numerous staking implementations including DappFactory, Team Finance, UNCX Network, UniCrypt, Geysr, Decubate, Add3, Houdini Swap Staking, Syncus, Yummy, GenesisDAO, and Synthetix. While each offers unique features, most suffer from either the gaming vulnerabilities described above or lack the flexibility to distribute arbitrary reward tokens. Synthetix's approach is particularly notable for its sophisticated reward calculation mechanisms, but it remains limited to protocol-specific tokens.

RainDrop's key innovation lies in combining temporal fairness (rewards only for pre-existing stakers) with cross-token flexibility and mathematical guarantees of proportional distribution.

5 Theorems

In this section, we prove that RainDrop distributes rewards efficiently and properly.

► **Theorem 1** (Constant-Time Operations). *All RainDrop protocol operations execute in constant time $O(1)$ with respect to the number of users and the number of historical drops.*

Proof. We analyze each operation's computational complexity:

- **Stake Operation σ :** Updates three state variables (S_{total} , $S_{user}(u)$, $E_{user}(u)$) using simple arithmetic operations and map lookups/updates, all of which are $O(1)$.
- **Unstake Operation μ :** Similar to stake, involves constant-time arithmetic and map operations.
- **Drop Operation δ :** Appends one element to the drops sequence D , increments counter k , and initializes tracking state. All operations are $O(1)$.
- **Claim Operation γ :** Computes a proportional share using arithmetic operations on stored values from the drop record. The key insight is that the formula $\frac{S_{user}(u) \times A_i}{S_i}$ uses only pre-stored values: $S_{user}(u)$ from the user map, and (A_i, S_i) from the immutable drop record d_i . No iteration over users or drops is required.
- **Withdraw Operation ζ :** Uses the running total C_i that is maintained incrementally during claim operations, avoiding the need to sum over all users. The remaining amount $R_i = A_i - C_i$ is computed in constant time.

Therefore, all operations achieve $O(1)$ complexity through careful state design and incremental maintenance of aggregate values. ◀

► **Theorem 2** (Proper Reward Distribution). *For any user u staking when drop d_i occurs, the user receives exactly their proportional share of the drop amount, and this share remains invariant under any sequence of actions by other users. Formally, if $S_{user}(u) > 0$ and $i > E_{user}(u)$ at the time of drop d_i , then for any sequence of actions $a_1, \dots, a_k$ performed by other users, the claim amount satisfies:*

$$\gamma(u, i) = \frac{S_{user}(u) \times A_i}{S_i}$$

where $S_{user}(u)$ and S_i are the user's stake and total stake at the time drop d_i was created.

Proof. Proof by induction on the sequence of actions $a_1, \dots, a_k$ performed after drop d_i :

- **Base case ($k = 0$):** Immediately after drop d_i is created, by definition of the claim operation:

$$\gamma(u, i) = \frac{S_{user}(u) \times A_i}{S_i}$$

where $S_{user}(u)$ and S_i are recorded at the time of drop creation.

- **Inductive step:** Assume the claim amount is preserved after actions $a_1, \dots, a_k$. We show it remains preserved after action a_{k+1} by case analysis:

- **Case 1 - Stake by another user $v \neq u$:** The stake operation updates S_{total} and $S_{user}(v)$ but does not modify the drop record $d_i = (A_i, \tau_i, S_i, T_i, C_i)$. Since $\gamma(u, i)$ depends only on the immutable values A_i , S_i , and the user's stake at drop time, it remains unchanged.
- **Case 2 - Unstake by another user $v \neq u$:** Similar to Case 1, unstaking modifies current state but not the historical drop record d_i . The claim amount $\gamma(u, i)$ remains invariant.

- **Case 3 - Claim by another user $v \neq u$:** The claim operation is a pure function that computes a value from the vault state without modifying it. It does not affect drop records or other users' potential claims.
- **Case 4 - New drop by any user:** Creating a new drop d_j with $j \neq i$ does not modify the existing drop record d_i , preserving $\gamma(u, i)$.
- **Case 5 - Withdraw remaining by drop creator:** This operation only affects unclaimed tokens after the deadline and does not modify the drop record d_i or affect valid claims made before the deadline.

In all cases, the drop record d_i containing $(A_i, \tau_i, S_i, T_i, C_i)$ remains immutable, and user u 's claim amount is preserved:

$$\gamma(u, i) = \frac{S_{user}(u) \times A_i}{S_i}$$

Therefore, users are guaranteed to receive exactly their proportional share of any drop they were eligible for, regardless of subsequent actions by other participants. ◀

6 Discussion

The RainDrop protocol provides a foundation for efficient and proper distribution of rewards to token holders. The temporal fairness mechanism prevents gaming of the system while encouraging long-term participation. The flexibility to distribute any fungible token as rewards opens possibilities for diverse incentive structures and community-driven tokenomics. The minimalism of RainDrop makes it suitable for composition with other protocols in decentralized systems. Most importantly, RainDrop's mathematical soundness and rigour paves the way for adoption of blockchain-based dividend distribution across the economy.

References

- 1 Add3. Website. <https://add3.io/>.
- 2 Announcing UNCX (formerly UniCrypt network). Medium. <https://medium.com/@UNCX/announcing-uncx-formerly-unicrypt-network-f9b41dbf82f1>.
- 3 Dappfactory. Project page. <https://medium.com/@dappfactory>.
- 4 Decubate. Documentation. <https://docs.decubate.com/>.
- 5 Genesisdao. Website. <https://genesisdao.com/>.
- 6 Houdini swap staking program. Documentation. <https://docs.houdiniswap.com/staking-program>.
- 7 Pancakeswap masterchef v2. BscScan. <https://bscscan.com/address/0xa5f8C5Dbd5F286960b9d90548680aE5ebFf07652#code>.
- 8 Sushiswap masterchef v1. Etherscan. <https://etherscan.io/address/0xc2edad668740f1aa35e4d8f227fb8e17dca888cd#code>.
- 9 Syncus. Website. <https://app.syncus.fi/>.
- 10 Synthetix. Website. <https://synthetix.io/>.
- 11 Team finance. Website. <https://team.finance/>.
- 12 Token geyser. Ampleforth Forum. <https://forum.ampleforth.org/t/token-geyser-2-0-phase-1/1027>.
- 13 Uncx network. Website. <https://uncx.network/>.
- 14 Yummy whitepaper. Website. <https://www.yummy-crypto.com/whitepaper>.
- 15 The MystenLabs Team. The Sui Smart Contracts Platform. Whitepaper. <https://docs.sui.io/paper/sui.pdf>.
- 16 Joachim Zahnentferner. hodlCoin: A financial game. Cryptology ePrint Archive, Paper 2023/1029, 2023.

A License

This work is licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International license². For derivative and commercial uses, contact the authors first.

B Official Implementations and Deployments of RainDrop

<https://github.com/StabilityNexus>

C Official RainDrop Channels

- <https://x.com/StabilityNexus>
- <https://discord.gg/7jS9qJNjJv>

² <https://creativecommons.org/licenses/by-nc-nd/4.0/>